

Commandes Git Utiles

Voici un guide des commandes Git les plus courantes pour gérer vos dépôts locaux et distants (GitHub, GitLab), avec description et exemples.

Tableau de référence des commandes indispensables Git

Commande	Usage	Description	Exemple
git add	- git add [nom_fichier] - git add .	- Ajoute un fichier à l'index (prépare pour le commit) - Ajoute tous les fichiers dans le dossier courant à l'index	- git add index.html -git add .
git commit	git commit -m "message"	Crée un commit avec un message décrivant les modifications	git commit -m "Correction bug login"
git push	git push	Envoie vos commits locaux vers le dépôt distant	git push

Définition des commandes indispensables

Si on souhaite ajouter tous les fichiers modifiés avant un commit (le plus pratique), et si notre branche locale est déjà liée à une branche distante (sur GitHub/GitLab) :

```
git add .  
git commit -m "message du commit"  
git push
```

git add . On sélectionne les fichiers qu'on veut "préparer" pour le commit.

Le point signifie "tous les fichiers modifiés, supprimés ou nouvellement créés dans le dossier courant et ses sous-dossiers".

En local seulement : ça touche uniquement notre dépôt (repository) sur notre ordinateur.

git commit -m "Message du commit" On crée un snapshot (une copie instantanée, une photo de notre code à ce moment).

Le message aide à savoir ce que contient ce commit.

En local seulement : notre dépôt distant n'est pas encore touché.

git push (ou git push -u origin nom_branche si la branche n'est pas encore liée à sa remote) Ici, on envoie nos commits depuis notre dépôt local vers le dépôt distant (GitHub/GitLab).

origin : c'est le nom par défaut de notre dépôt distant, configuré quand on a cloné notre projet.

-u : signifie "set upstream" ; associe la branche locale à la branche distante pour les futurs git push et git pull.

nom_de_la_branche : c'est la branche sur laquelle on veut envoyer nos commits (main ou master si on ne travaille pas encore sur d'autres branches, ou une branche spécifique pour chaque fonctionnalité sur laquelle on travaille).

Si on est en local sur une branche appelée "feature-login", et qu'une branche distante du même nom sur GitHub/GitLab existe aussi, faire **git push -u origin feature-login** va envoyer nos commits directement sur cette branche. Si cette branche n'existe pas sur Github/GitLab, la commande **git push -u origin feature-login** va créer automatiquement une nouvelle branche distante appelée feature-login, et les commits locaux qu'on a fait seront envoyés vers cette nouvelle branche distante.

Tableau de référence des commandes principales Git

Commande	Usage	Description	Exemple
git clone	git clone [URL]	Clone un dépôt distant sur votre machine et configure automatiquement le remote origin	git clone https://gitlab.com/user/projet.git

Commande	Usage	Description	Exemple
<code>git fetch</code>	- <code>git fetch</code> - <code>git fetch origin</code>	- Récupère les branches et commits distants sans fusionner et met à jour les références distantes (origin/main, origin/dev, etc.) sans modifier la branche locale - Récupère uniquement le remote origin (commandes souvent équivalentes car souvent un seul remote, qui est origin)	- <code>git fetch</code> - <code>git fetch origin</code>
<code>git status</code>	<code>git status</code>	Affiche l'état des fichiers (modifiés, ajoutés, non suivis) et indique si la branche locale est en avance ou en retard par rapport à sa branche distante	<code>git status</code>
<code>git pull</code>	- <code>git pull</code> - <code>git pull origin [nom_branche]</code>	- Récupère et fusionne automatiquement les commits depuis la branche remote du dépôt distant dans la branche courante (c'est l'équivalent d'un <code>git fetch</code> suivi d'un <code>git merge</code>) - Fait <code>fetch origin</code> puis <code>merge</code> avec la branche remote appelée (commandes souvent équivalentes)	- <code>git pull</code> - <code>git pull origin main</code>
<code>git init</code>	<code>git init</code>	Initialise un nouveau dépôt Git local dans le dossier courant	<code>git init</code>

Commande	Usage	Description	Exemple
<code>git remote add origin</code>	<code>git remote add origin [URL]</code>	Ajoute un dépôt distant nommé "origin" à votre dépôt local (cela ne rend pas votre dépôt local accessible sur GitHub/GitLab, mais configure seulement un lien vers un dépôt distant existant déjà créé sur GitHub/GitLab)	<code>git remote add origin https://gitlab.com/user/projet.git</code>
<code>git checkout -b</code>	<code>- git checkout -b [nom_branche]</code> <code>- git checkout -b [nom_branche] origin/[nom_branche]</code>	- Crée une nouvelle branche à partir de la branche actuelle puis se positionne dessus - Après un fetch, crée une branche locale à partir d'une branche distante et s'y positionne	<code>- git checkout -b feature-login</code> <code>- git checkout -b feature-login origin/feature-login</code>
<code>git checkout</code>	<code>git checkout [nom_branche]</code>	Change de branche	<code>git checkout develop</code>
<code>git branch</code>	<code>- git branch</code> <code>- git branch [nom_branche]</code>	- Liste les branches locales - Crée une nouvelle branche à partir de la branche actuelle sans se positionner dessus (donc en restant sur la branche actuelle)	<code>- git branch</code> <code>- git branch dev</code>
<code>git branch -m</code>	<code>git branch -m [ancien_nom] [nouveau_nom]</code>	Renomme une branche locale	<code>git branch -m develop dev</code>

Commande	Usage	Description	Exemple
<code>git push origin</code>	<code>- git push</code> <code>-u origin</code> <code>[nom_branche]</code> <code>- git push</code> <code>origin</code> <code>[nom_branche]</code>	- Pousse (envoie) les commits locaux vers le dépôt distant et définit la branche upstream (la branche remote associée) pour les futurs push/pull - Pousse les commits de la branche locale vers la branche remote spécifiée (même si ce n'est pas son upstream)	<code>- git push -u origin</code> <code>feature-login</code> <code>- git push origin dev</code>
<code>git merge</code>	<code>- git merge</code> <code>[nom_branche]</code> <code>- git merge</code> <code>origin/</code> <code>[nom_branche]</code>	- Fusionne une branche dans la branche courante - Fusionne une branche remote dans la branche courante	<code>- git merge feature-login</code> <code>- git merge origin/feature-signup</code>
<code>git log</code>	<code>- git log</code> <code>- git log</code> <code>--all</code> <code>- git log</code> <code>--oneline</code>	- Affiche l'historique des commits de la branche courante - Affiche l'historique des commits de toutes les branches - Affiche l'historique des commits sous une forme compacte : un commit par ligne avec son hash abrégé et son message. Plus lisible que <code>git log</code> pour avoir une vue rapide de l'historique	<code>- git log</code> <code>- git log --all</code> <code>- git log --oneline</code>
<code>git branch -r</code>	<code>git branch -r</code>	Liste toutes les branches distantes que le dépôt local connaît	<code>git branch -r</code>

Commande	Usage	Description	Exemple
<code>git branch -d</code>	<code>git branch -d [nom_branche]</code>	Supprime une branche locale (uniquement si elle a été entièrement fusionnée dans la branche courante ou dans sa branche remote)	<code>git branch -d feature-login</code>
<code>git branch -vv</code>	<code>git branch -vv</code>	Permet de voir l'upstream (la branche distante associée à la branche locale) : liste toutes les branches locales, leurs branches remotés associées, leur statut ahead/behind et leur dernier commit	<code>git branch -vv</code>

Tableau de référence des commandes secondaires Git

Commande	Usage	Description	Exemple
<code>git diff</code>	<code>git diff [nom_fichier]</code>	Montre les différences entre fichiers modifiés et dernier commit	<code>git diff fichier.txt</code>
<code>git diff</code>	<code>git diff [nom_branche1] [nom_branche2]</code>	Montre les différences entre deux branches	<code>git diff main develop</code>
<code>git switch</code>	<code>git switch [nom_branche]</code>	Change de branche (alternative à checkout)	<code>git switch develop</code>
<code>git switch -c (local)</code>	<code>git switch -c [nom_branche]</code>	Crée une nouvelle branche et s'y positionne	<code>git switch -c feature-signup</code>

Commande	Usage	Description	Exemple
<code>git switch -c</code> (remote)	<code>git switch -c [nom_branche]</code> origin/ [nom_branche]	Après un fetch, crée une branche locale à partir d'une branche distante et s'y positionne	<code>git switch -c feature-login</code> origin/feature-login
<code>git branch -D</code> ou <code>--delete</code>	<code>git branch -D [nom_branche]</code>	Supprime une branche locale même si non fusionnée	<code>git branch -D feature-login</code>
<code>git push origin --delete</code>	<code>git push origin --delete [nom_branche]</code>	Supprime une branche sur le dépôt distant (GitHub ou GitLab). Ne supprime pas la branche locale	<code>git push origin --delete feature-login</code>
<code>git remote -v</code>	<code>git remote -v</code>	Affiche les dépôts distants configurés (GitHub, GitLab, etc.)	<code>git remote -v</code>
<code>git stash</code>	<code>git stash</code>	Sauvegarde temporairement les modifications non commitées pour nettoyer le workspace	<code>git stash</code> <code>git stash pop</code> (pour récupérer les modifications) <code>git stash list</code> (pour lister les modifications)

Commande	Usage	Description	Exemple
<code>git rebase</code>	- <code>git rebase [nom_branche]</code> - <code>git rebase --continue</code> - <code>git rebase --abort</code> - <code>git rebase --skip</code>	- Réécrit l'historique des commits : rejoue les commits de la branche courante au-dessus d'une autre branche. Permet de récupérer les derniers changements tout en conservant un historique linéaire (sans commit de merge) - Après avoir résolu les conflits d'un rebase, indique à Git de poursuivre le rebase en appliquant le commit suivant - Annule complètement le rebase en cours et restaure la branche dans l'état exact où elle se trouvait avant le lancement du rebase - Ignore le commit actuellement en cours de rebase (souvent celui qui provoque un conflit) et passe directement au commit suivant	- <code>git rebase origin/dev</code> - <code>git rebase --continue</code> - <code>git rebase --abort</code> - <code>git rebase --skip</code>

Commande	Usage	Description	Exemple
<code>git push</code> <code>--force-with-lease</code>	<code>git push</code> <code>--force-with-lease</code>	Force la mise à jour de la branche distante après une réécriture de l'historique (par exemple après un rebase). Plus sûr que <code>--force</code> car il vérifie que personne n'a poussé de nouveaux commits sur la branche distante entre-temps	<code>git push</code> <code>--force-with-lease</code>
<code>git restore</code>	<code>- git restore [nom_fichier]</code> <code>- git restore .</code>	- Annule les modifications non indexées (non ajoutées avec <code>git add</code>) d'un fichier et le remet dans l'état du dernier commit - Annule toutes les modifications non indexées du dossier courant et de ses sous-dossiers	<code>- git restore app.js</code> <code>- git restore .</code>
<code>git restore --staged</code>	<code>- git restore --staged [nom_fichier]</code> <code>- git restore --staged .</code>	- Retire un fichier de l'index (annule le <code>git add</code>) sans perdre les modifications effectuées dans le fichier - Retire tous les fichiers de l'index sans perdre leurs modifications	<code>- git restore --staged app.js</code> <code>- git restore --staged .</code>

Commande	Usage	Description	Exemple
<code>git revert</code>	<code>git revert</code> <code>[hash_commit]</code>	Crée un nouveau commit qui annule les modifications introduites par un commit existant, sans réécrire l'historique. Recommandé pour annuler un commit déjà partagé sur GitHub/GitLab	<code>git revert</code> <code>a1b2c3d</code>

Bonnes pratiques de travail

GitLab

Cas 1 : Code récupéré sur GitLab :

- Sur GitLab, dans le projet : cliquer sur “Code” (en bleu à droite), puis sur l’image à droite de “Clone with HTTP” (“Copy URL”).
- Dans le terminal, se placer là où on souhaite déposer son projet, puis taper la commande `git clone <url-du-repo>` (ne pas oublier de rajouter un `-s` après `http`).
- Rentrer son username et son mot de passe GitLab.
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push` si on est sur la branche master en local.
- Pour travailler sur une autre branche pour une autre fonctionnalité : `git checkout -b <nouvelle-branche>` (ou `git branch <nouvelle-branche>` puis `git checkout <nouvelle-branche>` pour se positionner dessus).
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` si on a créé une nouvelle branche en locale et qu’on souhaite la créer aussi sur GitLab (utiliser le même nom de branche en local et sur GitLab).
- Merge Request sur GitLab avec merge entre branches (d’une branche source vers une branche cible), et si besoin plus tard : `git pull` en local.

Cas 2 : Projet créé en local puis poussé sur GitLab :

Sur GitLab :

- Cliquer sur “New Project” (en bleu à droite).

- Cliquer sur “Create blank project”.
- Remplir “Project name”.
- Dans “Project URL”, choisir son nom ou son groupe.
- Laisser en “Private”.
- Décocher “Initialize repository with a README”.
- Cliquer sur “Create project”.
- Une fois le projet créé, récupérer l’URL du projet en copiant ce qu’il y a après “git clone” (dans “Create a repository”).

En local dans le terminal :

- Taper la commande : `git init`, puis la commande `git remote add origin <url-du-repo>` (ne pas oublier de rajouter un `-s` après `http`).
- Taper ensuite les commandes `git add .` ou `git add <fichier(s)-modifié(s)>`
→ `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` (si vous êtes sur la branche master en local, il est recommandé d’appeler votre branche distante master aussi pour ne pas se perdre).
- Pour travailler sur une autre branche pour une autre fonctionnalité : `git checkout -b <nouvelle-branche>` (ou `git branch <nouvelle-branche>` puis `git checkout <nouvelle-branche>` pour se positionner dessus).
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"`
→ `git push -u origin <nom-branche>` si on a créé une nouvelle branche en locale et qu’on souhaite la créer aussi sur GitLab (utiliser le même nom de branche en local et sur GitLab).
- Merge Request sur GitLab avec merge entre branches (d’une branche source vers une branche cible), et si besoin plus tard : `git pull` en local.

GitHub

Cas 1 : Code récupéré sur GitHub :

- Sur GitHub, dans le dépôt : cliquer sur “Code” (bouton vert en haut à droite), puis copier l’URL dans l’onglet HTTPS ou SSH.
- Dans le terminal, se placer là où on souhaite déposer son projet, puis taper la commande `git clone <url-du-repo>`.
- En HTTPS, s’authentifier avec son compte GitHub (généralement via un token personnel ou le navigateur). En SSH, s’authentifier via une clé SSH préalablement configurée sur GitHub.
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"`
→ `git push` si on est sur la branche main ou master en local.
- Pour travailler sur une autre branche pour une autre fonctionnalité : `git checkout -b <nouvelle-branche>` (ou `git branch`

<nouvelle-branche> puis `git checkout <nouvelle-branche>` pour se positionner dessus).

- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` si on a créé une nouvelle branche en local et qu'on souhaite la créer aussi sur GitHub (utiliser le même nom de branche en local et sur GitHub).
- Pull Request sur GitHub avec merge entre branches (d'une branche source vers une branche cible), et si besoin plus tard : `git pull` en local.

Cas 2 : Projet créé en local puis poussé sur GitHub :

Sur GitHub :

- Cliquer sur votre photo de profil puis "Repositories" (dans le menu).
- Cliquer sur "New" (bouton vert en haut à droite).
- Choisir un propriétaire (soi-même ou une organisation) dans "Owner".
- Remplir "Repository name".
- Rajouter une courte description du projet.
- Choisir éventuellement "Public" ou "Private" dans "Choose visibility".
- Laisser "Start with a template" décoché en "No template".
- Laisser "Add README" décoché.
- Laisser également "Add .gitignore" et "Add license" décoché si le projet existe déjà en local.
- Cliquer sur "Create repository".
- Une fois le dépôt créé, récupérer l'URL du dépôt en copiant ce qu'il y a dans "Quick setup" (HTTPS ou SSH).

En local dans le terminal :

- Taper la commande : `git init`, puis la commande `git remote add origin <url-du-repo>`.
- Taper ensuite les commandes `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` (si vous êtes sur la branche master ou main en local, il est recommandé d'appeler votre branche distante de la même façon pour ne pas se perdre).
- Pour travailler sur une autre branche pour une autre fonctionnalité : `git checkout -b <nouvelle-branche>` (ou `git branch <nouvelle-branche>` puis `git checkout <nouvelle-branche>` pour se positionner dessus).
- Modification des fichiers en local, puis : `git add .` ou `git add <fichier(s)-modifié(s)>` → `git commit -m "<type-de-modification>"` → `git push -u origin <nom-branche>` si on a créé une nouvelle branche en local et qu'on souhaite la créer aussi sur GitHub (utiliser le même nom de branche en local et sur GitHub).

- Pull Request sur GitHub avec merge entre branches (d'une branche source vers une branche cible), et si besoin plus tard : `git pull` en local.

Cas de travail en groupe (pour GitHub et GitLab) :

Lorsqu'un autre développeur a fusionné sa branche dans dev ou main pendant que vous travaillez sur votre branche, on a deux choix :

Choix 1 : Synchronisation via une Pull Request / Merge Request

- Finir ses modifications, puis `git add .` et `git commit -m "mon message de commit"`
- `git push origin ma-branche`
- Ouvrir une PR/MR vers dev
- GitHub/GitLab détecte les conflits éventuels avec dev
- Résoudre les conflits directement dans l'interface
- Fusionner la branche une fois les conflits résolus sur GitHub/GitLab
- Plus tard, récupérer les changements de la branche remote origin/dev dans la branche locale dev (`git checkout dev` → `git pull` ou `git pull origin dev`).

Choix 2 : Synchronisation locale avant de pousser

- `git fetch origin`
- `git rebase origin/dev` ou `git merge origin/dev`
- Résoudre les éventuels conflits dans le terminal ou sur VSCode
- `git add .` → `git commit -m "mon message de commit"` → `git push` (ou `git push --force-with-lease` après un rebase)